亞洲大學
ASIA UNIVERSITY

Midterm Project Report

Advanced Computer Programming

Student Name : Nyamdavaa Baasanjav

Student ID : 114021164

Teacher : DINH-TRUNG VU

2024-04

Chapter 1 Introduction

1.1 Github

1) **Personal Github Account**: https://github.com/nyamkabaaska1-prog
2) **Group Project Repository**: https://github.com/nyamkabaaska1-prog/Group-1

1.2 Overview

The objective of this midterm project was to implement a robust web crawling system using the **Scrapy framework** to extract repository data from a specific GitHub profile . The system was designed to perform "Deep Scraping," navigating from a primary user profile page into individual repository sub-pages to capture comprehensive metadata .

Target Repository Source:

- Username: trungvdhp.
- Target URL: https://github.com/trungvdhp?tab=repositories.

Key advanced features and libraries utilized include:

- Scrapy 2.15.0: Leveraged for its asynchronous processing capabilities and built-in CSS selector engine.
- Item Pipelines: Utilized for automated data cleaning, specifically for date normalization and null-value handling.
- python-dateutil: Integrated within the pipeline to parse ISO-8601 timestamps into human-readable formats.
- Feed Exporters: Configured to generate structured XML output complying with specific midterm data requirements.

The program successfully extracted the URL, About, Last Updated, Languages, and Number of Commits for all repositories under the target profile, handling edge cases such as empty repositories and missing descriptions.

Chapter 2 Implementation

2.1 Class 1: GithubItem

Description: This class serves as the structured data container (Schema) for the project. It inherits from scrapy.Item to allow for seamless data flow between the Spider and the Pipeline.

2.1.1 Fields

- **url**: The absolute URL of the repository.
- **name**: The title of the repository.
- **about**: The description of the project, which includes a fallback logic to use the name if the description is empty.
- **lastUpdated**: The formatted timestamp of the most recent repository activity.
- **languages**: The primary programming language used.
- **commits**: The total count of commits extracted from the repository's detail page.

```
import scrapy

class GithubItem(scrapy.Item):
    url = scrapy.Field()
    name = scrapy.Field()
    about = scrapy.Field()
    lastUpdated = scrapy.Field()
    languages = scrapy.Field()
    commits = scrapy.Field()
```

Figure 1. GithubItem class definition in items.py

2.1.2 Methods

- No custom methods were implemented in this class as it serves strictly as a data schema .

2.2 Class 2: GithubSpider

Description: Located in spiders/github_spider.py, this class contains the crawling logic and instructions for navigating the GitHub DOM .

2.3 Method/Function 1: parse(self, response)

Description: This is the default callback for start_urls. It iterates through the repository list on the main profile page.

Logic: It extracts initial data (name, URL, about) and then generates a scrapy.Request for each repository's specific URL to find deeper metadata .

2.4 Method/Function 2: parse_details(self, response)

Description: This is the custom callback function used for deep scraping individual repository pages .

Logic: It extracts the commit count and handles the assignment logic for empty repositories. If no commits or languages are found, it sets those values to "None" per the assignment requirements.

```
def parse_details(self, response):
    item = response.meta['item']
    # Target the commit count on the specific repo page
    commits = response.css('span.d-none.d-sm-inline strong::text').get()

    # Empty repo logic
    if commits:
        item['commits'] = commits.strip()
    else:
        item['commits'] = "None"
        if not item['languages']:
            item['languages'] = "None"

    yield item
```

Figure 2. Implementation of parse_details with fallback logic .

2.5 Method/Function 3: GithubPipeline.process_item(self, item, spider)

Description: Located in pipelines.py, this method intercepts each item before it is exported .

Logic: It normalizes the lastUpdated date into a readable string and ensures all required fields meet the final XML formatting standards .

```
from dateutil import parser

class GithubPipeline:
    def process_item(self, item, spider):
        if item.get('lastUpdated'):
            date_obj = parser.parse(item['lastUpdated'])
            item['lastUpdated'] = date_obj.strftime('%d %B %Y, at %H:%M.')
        return item
```

Figure 3. Date normalization logic in pipelines.py .

Chapter 3 Results

3.1 Result 1: Console Output

The crawler successfully executed without errors, bypassing robots.txt via settings adjustments. It processed 24 repositories from the target URL.

```
 'item_scraped_count': 24,
 'items_per_minute': 45.0,
 'log_count/DEBUG': 49,
 'log_count/INFO': 3,
 'request_depth_max': 1,
 'response_received_count': 25,
 'responses_per_minute': 46.875,
 'scheduler/dequeued': 25,
 'scheduler/dequeued/memory': 25,
 'scheduler/enqueued': 25,
 'scheduler/enqueued/memory': 25,
 'start_time': datetime.datetime(2026, 4, 16, 6, 26, 53, 766189, tzinfo=datetime.timezone.utc)}
2026-04-16 14:27:26 [scrapy.core.engine] INFO: Spider closed (finished)
```

Figure 4. Terminal execution log showing successful item count .

3.2 Result 2: repositories.xml

The generated XML file contains structured data for all repositories. Each entry includes the five mandatory fields, with the "About" section correctly falling back to the repository name where descriptions were missing.


```
<?xml version="1.0" encoding="utf-8"?>
<items>
<item><name>caro-game-svn</name><url>https://github.com/trungvdhp/caro-game-svn</url><about>Automatically exported from code.google.com/p/caro-game-svn</about><languages>C#<
<item><name>qlkho-dongluc</name><url>https://github.com/trungvdhp/qlkho-dongluc</url><about>Automatically exported from code.google.com/p/qlkho-dongluc</about><languages>C#<
<item><name>project-004</name><url>https://github.com/trungvdhp/project-004</url><about>Automatically exported from code.google.com/p/project-004</about><languages>C#</langu
<item><name>netsharing</name><url>https://github.com/trungvdhp/netsharing</url><about>Automatically exported from code.google.com/p/netsharing</about><languages>Java</langua
<item><name>rsa-cs</name><url>https://github.com/trungvdhp/rsa-cs</url><about>Automatically exported from code.google.com/p/rsa-cs</about><languages>C#</languages><lastUpdat
<item><name>cong-thong-tin-sv</name><url>https://github.com/trungvdhp/cong-thong-tin-sv</url><about>Automatically exported from code.google.com/p/cong-thong-tin-sv</about><l
<item><name>duongtinhduyen</name><url>https://github.com/trungvdhp/duongtinhduyen</url><about>Automatically exported from code.google.com/p/duongtinhduyen</about><languages>
<item><name>trieutapsinhvien</name><url>https://github.com/trungvdhp/trieutapsinhvien</url><about>Automatically exported from code.google.com/p/trieutapsinhvien</about><lang
<item><name>xulyanhso</name><url>https://github.com/trungvdhp/xulyanhso</url><about>Automatically exported from code.google.com/p/xulyanhso</about><languages>C#</languages><
<item><name>oi-acm</name><url>https://github.com/trungvdhp/oi-acm</url><about>Automatically exported from code.google.com/p/oi-acm</about><languages>C++</languages><lastUpda
<item><name>file-encryption</name><url>https://github.com/trungvdhp/file-encryption</url><about>Automatically exported from code.google.com/p/file-encryption</about><languag
<item><name>xulyanhdongian</name><url>https://github.com/trungvdhp/xulyanhdongian</url><about>Automatically exported from code.google.com/p/xulyanhdongian</about><languages>
<item><name>studen-portal</name><url>https://github.com/trungvdhp/studen-portal</url><about>Automatically exported from code.google.com/p/studen-portal</about><languages>C#<
<item><name>master-ntou</name><url>https://github.com/trungvdhp/master-ntou</url><about>Automatically exported from code.google.com/p/master-ntou</about><languages>PHP</lang
<item><name>joomla</name><url>https://github.com/trungvdhp/joomla</url><about>joomla</about><languages>None</languages><lastUpdated>22 March 2017, at 00:54.</lastUpdated><co
<item><name>Chatbot-VN</name><url>https://github.com/trungvdhp/Chatbot-VN</url><about>Chatbot-VN</about><languages>Python</languages><lastUpdated>29 April 2018, at 09:43.</l
<item><name>DeepSky</name><url>https://github.com/trungvdhp/DeepSky</url><about>Deep Sky</about><languages>None</languages><lastUpdated>10 October 2018, at 02:00.</lastUpdat
<item><name>mpsnet</name><url>https://github.com/trungvdhp/mpsnet</url><about>Mobile Palm Vein Recognition using Saturation channel and Deep Learning-MPSNet</about><language
<item><name>palmid</name><url>https://github.com/trungvdhp/palmid</url><about>Palm vein recognizer for smartphones</about><languages>None</languages><lastUpdated>07 June 202
<item><name>testing</name><url>https://github.com/trungvdhp/testing</url><about>testing</about><languages>None</languages><lastUpdated>17 February 2025, at 02:58.</lastUpdat
<item><name>ACP</name><url>https://github.com/trungvdhp/ACP</url><about>Repository for Advanced Computer Programming AU-1132</about><languages>None</languages><lastUpdated>2
<item><name>ImageCollector</name><url>https://github.com/trungvdhp/ImageCollector</url><about>Image Collector</about><languages>None</languages><lastUpdated>11 September 202
<item><name>yolo-gradio-demo</name><url>https://github.com/trungvdhp/yolo-gradio-demo</url><about>Demo for AI Web Apps: YOLO Object Detection on Gradio Framework</about><lan
<item><name>cup-bottle-classifier</name><url>https://github.com/trungvdhp/cup-bottle-classifier</url><about>Cup bottle classifier</about><languages>Python</languages><lastUp
</items>
```


Figure 5. Final generated XML dataset .

Chapter 4 Conclusions

This project was a solid lesson in how Scrapy handles the "heavy lifting" of web scraping compared to simpler tools like BeautifulSoup. It wasn't all smooth sailing-especially when the code initially got ghosted by GitHub's *robots.txt* file . Figuring out how to bypass those restrictions by adjusting the settings and using a proper pipeline to clean up the messy timestamps was a practical way to see the "Advanced" part of this course in action.

The final XML output proves the logic is working as intended. The spider successfully crawled all 24 repositories, applied the fallback name whenever a description was missing, and correctly tagged empty repositories with "None" as requested in the assignment notes . Overall, it's a functional piece of code that fulfills every requirement and handles the data how it was supposed to.

.

Chapter 5 References

Academic & Technical References

- **Hunt, J. (2019).** *Advanced Guide to Python 3 Programming* (2nd ed.). Springer.
- **Mitchell, R. (2018).** *Web Scraping with Python: Collecting More Data from the Modern Web* (2nd ed.). O'Reilly Media.
- **Scrapy Community. (2026).** *Scrapy Documentation (Version 2.15.0)*. Retrieved from https://docs.scrapy.org/

Source Data

- **GitHub, Inc. (2026).** *trungvdhp Public Repositories*. Retrieved from https://github.com/trungvdhp?tab=repositories.